CHAPTER 11

Handling Terraform Errors & Debugging

Diagnose syntax, validation, dependency, provider, and state errors — then fix them with fmt, validate, taint, and refresh.

Errors

Debug Logs

CLI Tools

Scenario

What We'll Cover

A practical path from spotting an error to a fixed, applied configuration.

11.1 Understanding Terraform Errors

Five common categories: syntax, validation, dependency, provider, state

11.2 Debugging with Logs

TRACE, DEBUG, INFO, WARN, ERROR — and how to capture them

11.3 Core CLI Commands

fmt · validate · taint · refresh

Scenario Walkthrough

Fixing an EC2 instance that won't create properly

11.1 Understanding Terraform Errors

Terraform errors generally fall into five categories:

Syntax Errors

Incorrect HCL syntax — e.g. missing { } or =

Validation Errors

Invalid resource names, unsupported arguments, or bad values

Dependency Errors

Missing dependencies between resources

Provider Errors

Cloud authentication or configuration issues

State Errors

Corrupt or outdated state file

Example: Syntax Error

Missing "=" between argument name and value

Incorrect

```
resource "aws_instance" "example" {
  ami "ami-12345678"
  instance_type "t2.micro"
}
```

Terraform cannot parse the block — the argument value is ambiguous.

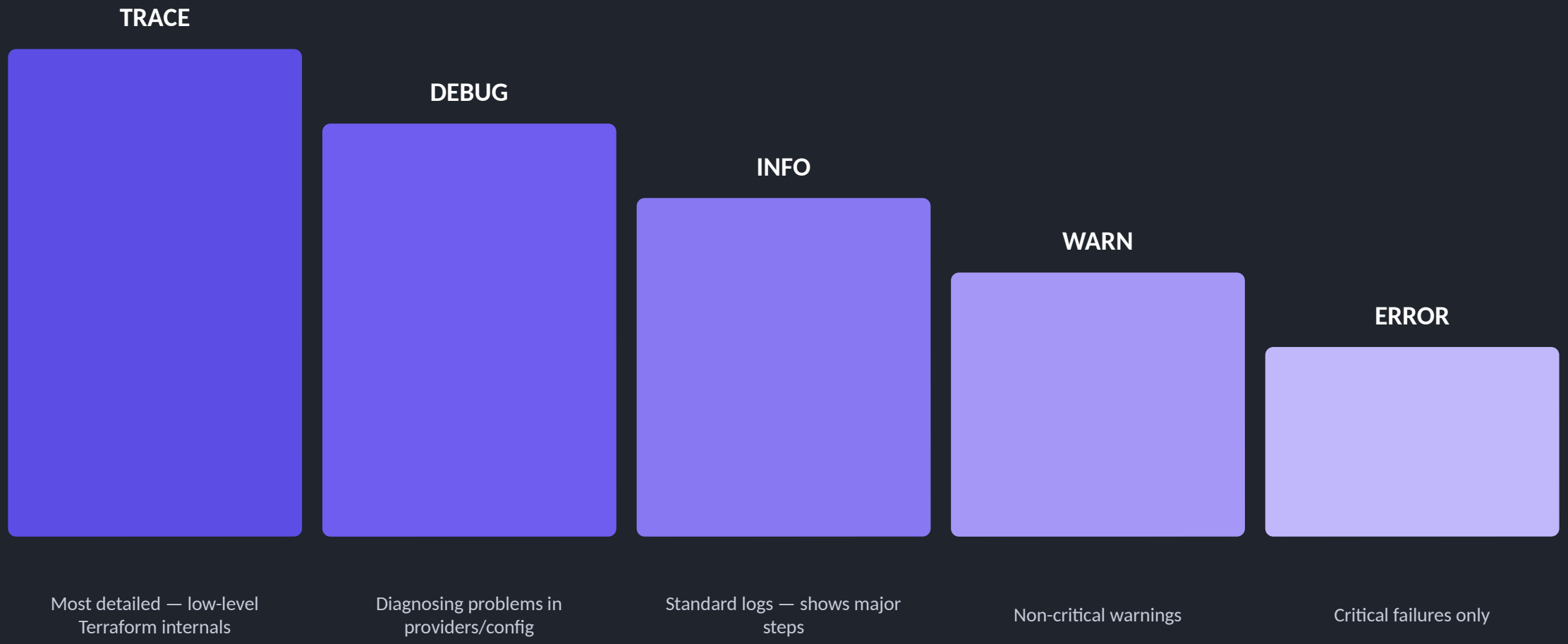
Fixed

```
resource "aws_instance" "example" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}
```

Adding "=" lets Terraform parse the file correctly and proceed to plan.

11.2 Debugging with Terraform Logs

Five verbosity levels, from most to least detailed:



Enabling & Saving Debug Logs

Print Debug Logs to Console

```
$ export TF_LOG=DEBUG
$ terraform apply
```

Terraform prints detailed debug logs directly to the terminal during the run.

Save Debug Logs to a File

```
$ export TF_LOG=DEBUG
$ export TF_LOG_PATH="terraform-debug.log"
$ terraform apply
```

All logs are saved to terraform-debug.log for later review or sharing with a team.

11.3 Core CLI Commands (1/2)

`terraform fmt`

Fix Formatting Issues

Automatically formats Terraform files to match best-practice style — spacing, indentation, alignment.

```
$ terraform fmt
```

✔️ removes unnecessary spaces and fixes indentation

`terraform validate`

Validate Configuration

Checks for syntax or logical errors without executing any changes to real infrastructure.

```
$ terraform validate
```

✔️ flags incorrect resource names, missing values, etc.

11.3 Core CLI Commands (2/2)

`terraform taint`

Force Resource Recreation

Marks a resource as tainted, forcing Terraform to destroy and recreate it on the next apply.

```
$ terraform taint aws_instance.example
$ terraform apply
```

✔️ destroys and recreates the EC2 instance

`terraform refresh`

Sync State with Reality

Updates the Terraform state file to match the actual live state of the infrastructure.

```
$ terraform refresh
```

✔️ useful when changes were made manually in the cloud console

Scenario: EC2 Instance Not Created

Troubleshooting a configuration step by step

The Problem

```
resource "aws_instance" "example" {
  ami           = "ami-12345678"
  instance_type = var.instance_size
}
```

The variable instance_size is referenced but never declared.

Step 1 · Run terraform validate

```
Error: Reference to undeclared input variable
"var.instance_size" not defined
```

Solution: declare the variable in variables.tf

```
variable "instance_size" {
  default = "t2.micro"
}
```

Scenario: Steps 2–4

Continuing the troubleshooting workflow to a successful apply

Step 2 · Run in Debug Mode

```
$ export TF_LOG=DEBUG
$ terraform apply
```

Error: Invalid AMI ID

Solution: use a valid AMI ID for the target region.

Step 3 · Force Recreate

```
$ terraform taint
aws_instance.example
$ terraform apply
```

✔️ **Terraform recreates the EC2 instance**

Step 4 · Refresh State

```
$ terraform refresh
```

✔️ **Syncs the state file with actual infrastructure**

Chapter 11 Summary

Five error types

Syntax, validation, dependency, provider, and state errors each need a different lens.

Debug logs

TF_LOG and TF_LOG_PATH surface and capture the details behind a failure.

fmt, validate, taint, refresh

Four commands that format, check, force-recreate, and reconcile state.

The scenario

validate → debug → taint → refresh fixed a broken EC2 deployment end to end.